Tema 6

Aspectos prácticos del uso de perceptrones multicapa.

6.1 Material de estudio

Los materiales de estudio de este tema son la segunda parte del capítulo 10 de Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition desde Implementing MLPs with Keras inclusive y adquirir un conocimiento de las herramientas que en ellos se comenta principalmente a partir de la documentación de Keras y TensorFlow. Iremos detallando el uso de estos materiales en las secciones siguientes.

Los códigos de ejemplo para este tema ya han sido descargados en temas anteriores.

6.2 Comentarios

6.2.1 Instalación de las herramientas necesarias

6.2.1.1 Instalación local

Para instalar las herramientas TensorFlow y Keras lo mejor es no seguir las indicaciones del libro. Sino iniciar Anaconda Navigator, con permisos de administrador (en Windows), hacer click en Enviroments, listar paquetes "no instalados" o "todos" y buscar "keras". En la lista aparecerán varios paquetes, los importantes son keras y keras-gpu. Si tenéis un ordenador con una GPU de NVIDIA con CUDA Compute Capacity >= 3.5 marcad keras-gpu, si no, marcad keras. A continuación haced click en el botón Apply que aparecerá abajo a la derecha. No es necesario marcar más paquetes, keras arrastrará todos los

demás que son necesarios entre ellos tensorflow y tensorboard. Para muchas de las tareas usuales es preferible, por la comodidad que aporta, usar Keras que usar TensorFlow directamente.

6.2.1.2 Ejecución en la nube

En este blob se da información de cómo usar aceleración por hardware mediante GPUs, también se pueden usar TPUs. Para los casos que nos ocupan el uso de GPUs es recomendable frente al de TPUs, ver comparativa. El uso de aceleración por hardware es muy recomendable frente a usar solo la CPU, se pueden conseguir reducciones de tiempo de 10 veces o incluso de hasta 50. Esto valores los he observado con ejecuciones en local, teniendo el hardware apropiado en el PC. Sin embargo puede ser que para que se note la mejora en la velocidad con GPUs o TPUs en Google Colab haya que usarlo por la noche que es cuando menos usuarios están usándolo en Europa, de día puede ser incluso más lento que usando CPU.

Google Colab también permite la ejecución en local instalándonos las herramientas necesarias.

6.2.1.3 Código de testeo de GPUs

Este código testea si TensorFlow detecta detecta nuestra(s) GPU(s) o las de Google Colab:

```
import tensorflow as tf
assert tf.__version__ >= "2.0"

gpus_list = tf.config.list_physical_devices('GPU')
if not gpus_list:
    print("No GPU was detected.")
else:
    print(tf.config.list_physical_devices('GPU'))
```

Si se usa TensorFlow 1.x

```
from tensorflow.python.client import device_lib
def get_available_gpus():
        local_device_protos = device_lib.list_local_devices()
        return [x.name for x in local_device_protos if x.
           device_type == 'GPU']
```

```
print("Available GPUs ")
get_available_gpus()
```

si el resultado es algo como ['/device:GPU:0'] es que la ha detectado, si hay más de una aparecerán :1, :2, etc.

6.2.1.4 Código para liberar GPUs

Puede ser conveniente incluir este código al final de los notebooks que usen tensorflow-gpu, si no puede ser que la memoria de la GPU no se libere hasta que se cierre Jupyter.

```
from numba import cuda
cuda.select_device(0)
cuda.close()
```

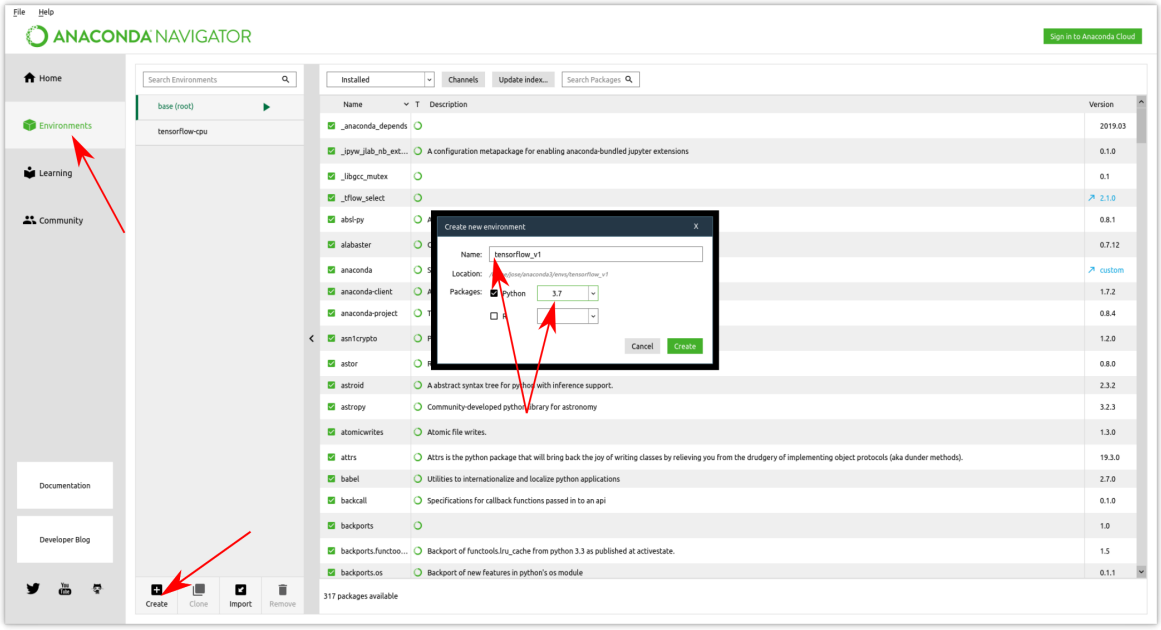
si hay más devices añadir cuda.select_device(1), etc.

Este código usa numba que deberemos tener instalado.

6.2.2 Instalación opcional de TensorFlow versión 1

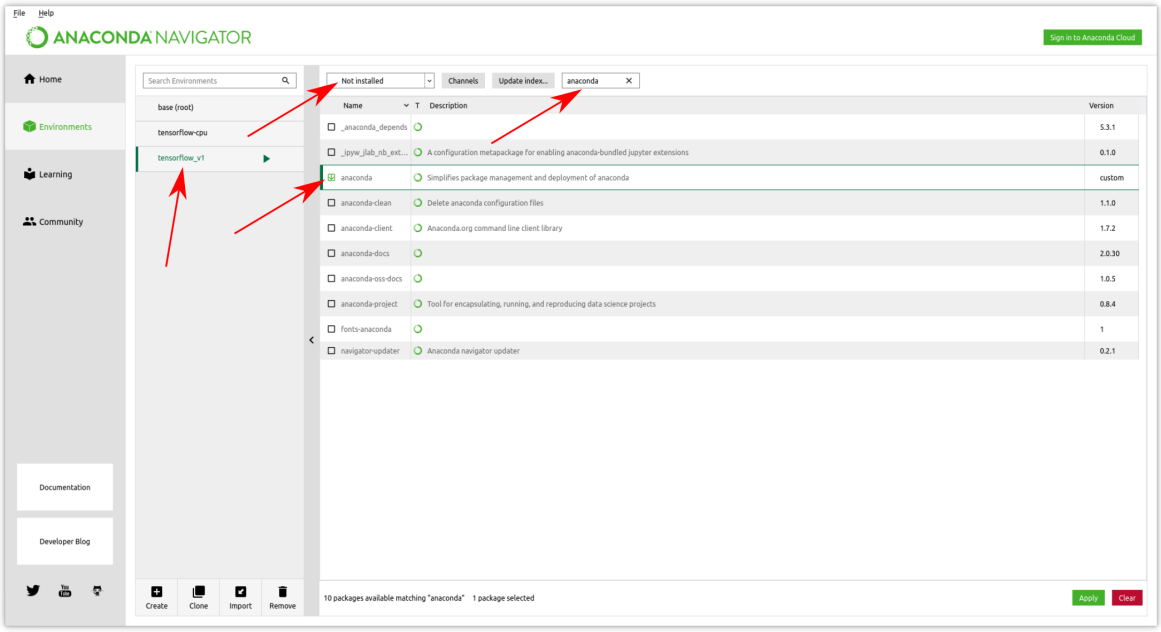
Con las instrucciones anteriores se instala TensorFlow en su versión 2 que es la que necesitamos para los notebooks del libro del curso y las actividades. Sin embargo existen bastantes diferencias entre la versión 2 y la 1 y bastantes de los ejemplos que se pueden encontrar en Internet hacen uso de la versión 1 hasta que se actualicen. Google digamos que está forzando el uso de la nueva versión y que esta tiene una integración más fluida con Keras, pero no hay una traducción directa del código de para adaptarlo a la nueva versión, consultar la guía de migración. Si se quiere ejecutar ejemplos que usen la versión 1 lo mejor es crear un nuevo entorno en Anaconda. Damos las instrucciones a continuación, hay que ejecutar Anaconda Navigator con permisos de administrador (en Windows):

1. Crear el nuevo entorno, al que denominaremos por ejemplo tensorflow_v1, instalando los paquetes básicos.



.

2. Instalar anaconda. Elegir el entorno, paquetes Not installed, buscar anaconda y click izquierdo sobre la marca a la izquierda del paquete. La instalación de anaconda arrastra muchos paquetes entre ellos matplotlib, jupyter, numpy, pandas, scipy, seaborn, scikit-learn.



.

3. De la misma forma instalar conda.

4. A continuación se instala tensorflow para cpu (paquete tensorflow a secas) o gpu (paquete tensorflow-gpu), las versiones pueden ser la 1.15.0 o la 1.14.0. Para ello

procedemos como con los paquetes anteriores pero hacemos click derecho sobre la marca a la izquierda del paquete y elegimos en el desplegable Mark for specific version installation y en el siguiente desplegable elegimos la versión.

5. A continuación instalamos directamente, igual que los demás que hemos instalado en 2. y 3., keras o keras-gpu, dependiendo de si hemos instalado tensorflow o tensorflow-gpu.

6. Opcionalmente si queremos que se muestren los gráficos de habrá que instalar pydot y pygraphviz, así como el propio Graphviz que no es un paquete Python. Esto también permite exportar los Árboles de Decisión.

6.2.3 Comentarios sobre el texto del libro

En la sección Building an Image Classifier Using the Sequential API se va a crear una red que clasifique fotos de prendas de vestir de MNIST usando el Sequential model de Keras, también hay que conocer su API. En el libro se crea el modelo, se compila, se entrena y evalúa y se usa para hacer predicciones. A continuación en la sección Building a Regression MLP Using the Sequential API se aplica el modelo secuencial para hacer predicciones mediante regresión.

En las siguientes secciones se comentan usos de la Functional API de Keras, cómo crear subclases de las APIs comentadas, cómo guardar y recuperar un modelo previamente entrenado, el uso de callbacks para un mejor control del proceso de entrenamiento. A continuación se comenta el uso de Tensorboard para visulización. Termina el texto con explicaciones de cómo ajustar los hiperparámetros de una red, los más importantes son el número de capas ocultas y el número de neuronas en cada capa oculta.

6.2.4 Comentarios sobre el notebook

En notebook asociado a este tema es 10_neural_nets_with_keras.ipynb. Conviene que ejecutar el notebook para comprobar que todo va bien. La celda que consumen más tiempo de ejecución es la 100, donde se usa RandomizedSearchCV. Si no usamos GPU, poniendo el parámetro n_jobs > 1 o con un valor negativo se aumenta la velocidad, ver enlace anterior.

6.3 Actividades autoevaluables

Realizar los ejercicios del 6 al 9 inclusive que se encuentran en la sección Exercices del capítulo 10 Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow. En el notebook asociado están las soluciones.

6.4 Actividades evaluables

La actividad evaluable va a consistir en realizar una versión diferente del ejercicio 10 del capítulo 10 del libro. En el libro se comenta que se entrene un MLP profundo, vamos a limitar este requisito ya que si no lo tiempos de ejecución se puede disparar especialmente si no se usa GPU. En el ejemplo del libro en la sección CREATING THE MODEL USING THE SEQUENTIAL API se construye una red con una capa oculta.

Este ejercicio aparece resuelto en el github por lo cual vamos a pedir que se realicen las siguientes modificaciones respecto a lo que se indica en el libro.

1. Hay que construir redes con al menos 3 y 4 capas ocultas.
2. Tanto para la red de 3 como para la de 4 capas ocultas hay que crear dos versiones con diferentes números de neuronas en las capas ocultas y comparar los resultados. Aquí se utilizará el factor de LearningRate que se utiliza en github.
3. Para las redes con 3 y 4 capas que hayan obtenido los mejores resultados en el apartado anterior cambiar el factor de LearningRate y comparar los resultados con los obtenidos en el apartado 2).

Recordad que no se va a evaluar estrictamente el alcanzar el 98% de precisión, basta con no quedarse muy lejos. Lo que sí hay que hacer es diseñar un procedimiento para buscar el learning rate apropiado, dibujar la función de pérdida, usar early stopping y las demás cuestiones que se piden en el ejercicio del libro.

Habrá que crear un notebook con celdas de tipo code y de tipo markdwon, en las de este último tipo se incluirán comentarios de lo que se esté haciendo en las de tipo code o sobre los resultados.

6.4.1 Valoración

Se valorará la creación del código que realice los requisitos enumerados, la presencia de comentarios de lo que hace el código y sus resultados y las comparaciones entre

los resultados obtenidos por las distintas redes, así como algún gráfico que muestre los resultados. Para terminar incluir un apartado de conclusiones. Tanto los comentarios como las conclusiones no es necesario que sea extensos, sino que describan de forma concisa. Dado que la parte de código está bastante resuelta en el github, se le dará bastante importancia en la corrección a los comentarios.

6.5 Otras actividades opcionales

Leer los artículos que se citan en el libro y en las notas al final del capítulo. Leer el apéndice E de la segunda edición de Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, sobre otros modelos históricos de redes neuronales. Para profundizar más en los aspectos matemáticos del tema leer el capítulo 11 de The Elements of Statistical Learning.